

apps/api/main.py

```
from future import annotations
```

```
import sys from pathlib import Path
```

```
from fastapi import FastAPI from fastapi.middleware.cors import CORSMiddleware from  
fastapi.staticfiles import StaticFiles
```

build_materials.py starts the server via:

```
python apps/api/main.py
```

When executed as a script, we need repo-root on sys.path so apps.*

```
imports work. REPO_ROOT = Path(file).resolve().parents[2] if str(REPO_ROOT) not in  
sys.path: sys.path.insert(0, str(REPO_ROOT))
```

```
from apps.api.core.db import init_db # noqa: E402 from apps.api.core.seed import  
ensure_seeded # noqa: E402 from apps.api.routers import ( # noqa: E402 assessments,  
audit, auth, datasets, indicators, knowledge, projects, quality, structures, )
```

```
def create_app() -> FastAPI: app = FastAPI(title="结构抗震韧性评估专用知识库与数据  
治理平台软件", version="1.0")
```

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"],  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

```
@app.get("/api/health")  
def health() -> dict[str, str]:  
    return {"status": "ok"}
```

```

app.include_router(auth.router, prefix="/api")
app.include_router(projects.router, prefix="/api")
app.include_router(structures.router, prefix="/api")
app.include_router(datasets.router, prefix="/api")
app.include_router(quality.router, prefix="/api")
app.include_router(indicators.router, prefix="/api")
app.include_router(assessments.router, prefix="/api")
app.include_router(knowledge.router, prefix="/api")
app.include_router(audit.router, prefix="/api")

return app

app = create_app()

init_db() ensure_seeded()

app.mount("/", StaticFiles(directory="apps/web", html=True), name="web")

if name == "main": import uvicorn

uvicorn.run("apps.api.main:app", host="127.0.0.1", port=8010, log_level=
"warning")

```

apps/api/routers/init.py

```

from future import annotations

```

router package marker

apps/api/routers/assessments.py

```

from future import annotations

```

```

import json import time

```

```

from fastapi import APIRouter, Depends from pydantic import BaseModel, Field

```

```

from ..core.authz import TokenClaims, require_perm from ..core.db import get_conn from
..services.audit import audit_write from ..services.lineage import
write_lineage_for_assessment

```

```

router = APIRouter(tags=["assessments"])

```

```

class AssessmentIn(BaseModel): structure_id: int summary: str = Field(min_length=5)
indicator_ids: list[int]

```

```

class ReviewIn(BaseModel): status: str note: str = ""

```

```

@router.get("/assessments") def list_assessments(structure_id: int, _: TokenClaims =
Depends(require_perm("assessments:read"))) -> list[dict]: with get_conn() as conn: rows =
conn.execute("SELECT * FROM assessments WHERE structure_id=? ORDER BY id
DESC", (structure_id,)).fetchall() return [dict(r) for r in rows]

```

```

@router.post("/assessments") def create_assessment(data: AssessmentIn, claims:
TokenClaims = Depends(require_perm("assessments:write"))) -> dict: with get_conn() as
conn: cur = conn.execute( "INSERT INTO assessments(structure_id, status, summary,
created_by, created_at) VALUES(?,?,?,?)", (data.structure_id, "submitted",
data.summary, claims.sub, int(time.time())) , ) assessment_id = int(cur.lastrowid)
write_lineage_for_assessment(conn, assessment_id, data.indicator_ids)

```

```

    audit_write(conn, claims.sub, "create", "assessment",

```

```

str(assessment_id), "submitted") return {"id": assessment_id}

```

```

@router.post("/assessments/{assessment_id}/review") def
review_assessment(assessment_id: int, data: ReviewIn, claims: TokenClaims =
Depends(require_perm("assessments:review"))) -> dict: if data.status not in {"approved",
"rejected"}: return {"ok": False, "message": "status must be approved/rejected"} with
get_conn() as conn: conn.execute( "UPDATE assessments SET status=?, reviewer=?,
reviewed_at=?, review_note=? WHERE id=?", (data.status, claims.sub, int(time.time()),
data.note, assessment_id), ) audit_write(conn, claims.sub, "review", "assessment",
str(assessment_id), f'status={data.status}') return {"ok": True}

```

apps/api/routers/audit.py

```

from future import annotations

```

```

from fastapi import APIRouter, Depends

```

```

from ..core.authz import TokenClaims, require_perm from ..core.db import get_conn

router = APIRouter(tags=["audit"])

@router.get("/audit/logs") def list_logs(limit: int = 200, _: TokenClaims =
Depends(require_perm("audit:read"))) -> list[dict]: limit = max(1, min(500, int(limit)))
with get_conn() as conn: rows = conn.execute("SELECT * FROM audit_logs ORDER BY
id DESC LIMIT ?", (limit,)).fetchall() return [dict(r) for r in rows]

```

apps/api/routers/auth.py

```

from future import annotations

from fastapi import APIRouter, Depends, HTTPException from pydantic import
BaseModel

from ..core.authz import require_user from ..core.db import get_conn from ..core.security
import TokenClaims, issue_token from ..services.audit import audit_log

router = APIRouter(tags=["auth"])

class LoginIn(BaseModel): username: str password: str

@router.post("/auth/login") def login(data: LoginIn) -> dict[str, str]: with get_conn() as
conn: row = conn.execute( "SELECT username, role, display_name FROM users WHERE
username=? AND password=?", (data.username, data.password), ).fetchone() if not row:
raise HTTPException(status_code=401, detail="bad credentials") token =
issue_token(subject=row["username"], role=row["role"]) audit_log(conn,
actor=row["username"], action="login", object_type= "user", object_id=row["username"],
detail="ok") return {"token": token, "role": row["role"], "display_name":
row["display_name"]}

@router.get("/auth/me") def me(claims: TokenClaims = Depends(require_user)) -> dict[str,
str]: return {"username": claims.sub, "role": claims.role}

```

apps/api/routers/datasets.py

```

from future import annotations

import json

```

```

from fastapi import APIRouter, Depends
from pydantic import BaseModel, Field

from ..core.authz import TokenClaims, require_perm
from ..core.db import get_conn
from ..services.audit import audit_write

router = APIRouter(tags=["datasets"])

class DatasetIn(BaseModel):
    name: str = Field(min_length=2)
    category: str
    source_desc: str
    sensitivity_level: str

class DatasetVersionIn(BaseModel):
    dataset_id: int
    version: str
    change_note: str
    payload_json: dict

@router.get("/datasets")
def list_datasets(_: TokenClaims = Depends(require_perm("datasets:read"))) -

    list[dict]:
        with get_conn() as conn:
            rows = conn.execute("SELECT * FROM datasets ORDER BY id DESC").fetchall()
            return [dict(r) for r in rows]

@router.post("/datasets")
def create_dataset(data: DatasetIn, claims: TokenClaims = Depends(require_perm("datasets:write"))) -> dict:
    with get_conn() as conn:
        cur = conn.execute(
            "INSERT INTO datasets(name, category, owner, source_desc, sensitivity_level) VALUES(?,?,?,?,?)",
            (data.name, data.category, claims.sub, data.source_desc, data.sensitivity_level),
        )
        audit_write(conn, claims.sub, "create", "dataset", str(cur.lastrowid), f"name={data.name}")
        return {"id": cur.lastrowid}

@router.get("/datasets/{dataset_id}/versions")
def list_versions(dataset_id: int, _: TokenClaims = Depends(require_perm("datasets:read"))) -> list[dict]:
    with get_conn() as conn:
        rows = conn.execute(
            "SELECT id, dataset_id, version, change_note, created_at FROM dataset_versions WHERE dataset_id=? ORDER BY id DESC",
            (dataset_id,)
        ).fetchall()
        return [dict(r) for r in rows]

@router.post("/datasets/versions")
def create_version(data: DatasetVersionIn, claims: TokenClaims = Depends(require_perm("datasets:write"))) -> dict:
    with get_conn() as conn:
        cur = conn.execute(
            "INSERT INTO dataset_versions(dataset_id, version, change_note, payload_json) VALUES(?,?,?,?)",
            (data.dataset_id, data.version, data.change_note, json.dumps(data.payload_json, ensure_ascii=False)),
        )
        audit_write(conn, claims.sub, "create", "dataset_version", str(cur.lastrowid), f"dataset_id={data.dataset_id}, v={data.version}")
        return {"id": cur.lastrowid}

```

apps/api/routers/indicators.py

```
from future import annotations
```

```
import json
```

```
from fastapi import APIRouter, Depends from pydantic import BaseModel, Field
```

```
from ..core.authz import TokenClaims, require_perm from ..core.db import get_conn from  
..services.audit import audit_write
```

```
router = APIRouter(tags=["indicators"])
```

```
class IndicatorIn(BaseModel): structure_id: int name: str = Field(min_length=2) method:  
str unit: str data_bindings_json: dict
```

```
@router.get("/indicators") def list_indicators(structure_id: int, _: TokenClaims =  
Depends(require_perm("indicators:read"))) -> list[dict]: with get_conn() as conn: rows =  
conn.execute("SELECT * FROM indicators WHERE structure_id=? ORDER BY id  
DESC", (structure_id,)).fetchall() return [dict(r) for r in rows]
```

```
@router.post("/indicators") def create_indicator(data: IndicatorIn, claims: TokenClaims =  
Depends(require_perm("indicators:write"))) -> dict: with get_conn() as conn: cur =  
conn.execute( "INSERT INTO indicators(structure_id, name, method, unit,  
data_bindings_json) VALUES(?,?,?,?,?)", (data.structure_id, data.name, data.method,  
data.unit, json.dumps(data.data_bindings_json, ensure_ascii=False)), ) audit_write(conn,  
claims.sub, "create", "indicator", str(cur.lastrowid), f"name={data.name}") return {"id":  
cur.lastrowid}
```

apps/api/routers/knowledge.py

```
from future import annotations
```

```
import json import time
```

```
from fastapi import APIRouter, Depends from pydantic import BaseModel, Field
```

```
from ..core.authz import TokenClaims, require_perm from ..core.db import get_conn from  
..services.audit import audit_write
```

```
router = APIRouter(tags=["knowledge"])
```

```
class KnowledgeIn(BaseModel): title: str = Field(min_length=4) tags: list[str] = []  
summary: str = "" evidence_refs_json: dict = {}
```

```
@router.get("/knowledge") def list_entries(q: str = "", _: TokenClaims =  
Depends(require_perm("knowledge:read"))) -> list[dict]: with get_conn() as conn: if  
q.strip(): rows = conn.execute( "SELECT * FROM knowledge_entries WHERE title LIKE  
? OR tags LIKE ? ORDER BY id DESC", (f"%{q}%", f"%{q}%", ), ).fetchall() else: rows  
= conn.execute("SELECT * FROM knowledge_entries ORDER BY id DESC").fetchall()  
return [dict(r) for r in rows]
```

```
@router.post("/knowledge") def create_entry(data: KnowledgeIn, claims: TokenClaims =  
Depends(require_perm("knowledge:write"))) -> dict: with get_conn() as conn: cur =  
conn.execute( "INSERT INTO knowledge_entries(title, tags, summary,  
evidence_refs_json, created_at, created_by) VALUES(?,?,?,?,?,?)", ( data.title,  
", ".join(data.tags), data.summary, json.dumps(data.evidence_refs_json,  
ensure_ascii=False), int(time.time()), claims.sub, ), ) audit_write(conn, claims.sub,  
"create", "knowledge_entry", str(cur.lastrowid), f"title={data.title}") return {"id":  
cur.lastrowid}
```

apps/api/routers/projects.py

```
from future import annotations
```

```
from fastapi import APIRouter, Depends from pydantic import BaseModel, Field
```

```
from ..core.authz import TokenClaims, require_perm from ..core.db import get_conn from  
..services.audit import audit_write
```

```
router = APIRouter(tags=["projects"])
```

```
class ProjectIn(BaseModel): name: str = Field(min_length=2) region: str site_class: str  
importance: str
```

```
@router.get("/projects") def list_projects(_: TokenClaims =  
Depends(require_perm("projects:read"))) -
```

```
list[dict]: with get_conn() as conn: rows = conn.execute("SELECT * FROM  
projects ORDER BY id DESC") .fetchall() return [dict(r) for r in rows]
```

```
@router.post("/projects") def create_project(data: ProjectIn, claims: TokenClaims = Depends(require_perm("projects:write"))) -> dict: with get_conn() as conn: cur = conn.execute( "INSERT INTO projects(name, region, site_class, importance, owner) VALUES(?,?,?,?)" , (data.name, data.region, data.site_class, data.importance, claims.sub), ) audit_write(conn, claims.sub, "create", "project", str(cur.lastrowid) , f"name={data.name}") return {"id": cur.lastrowid}
```

```
@router.put("/projects/{project_id}") def update_project(project_id: int, data: ProjectIn, claims: TokenClaims = Depends(require_perm("projects:write"))) -> dict: with get_conn() as conn: conn.execute( "UPDATE projects SET name=?, region=?, site_class=?, importance=? WHERE id=?", (data.name, data.region, data.site_class, data.importance, project_id), ) audit_write(conn, claims.sub, "update", "project", str(project_id), "fields=all") return {"ok": True}
```

```
@router.delete("/projects/{project_id}") def delete_project(project_id: int, claims: TokenClaims = Depends(require_perm("projects:write"))) -> dict: with get_conn() as conn: conn.execute("DELETE FROM projects WHERE id=?", (project_id,)) audit_write(conn, claims.sub, "delete", "project", str(project_id), "ok") return {"ok": True}
```

apps/api/routers/quality.py

```
from future import annotations
```

```
import json import time
```

```
from fastapi import APIRouter, Depends from pydantic import BaseModel
```

```
from ..core.authz import TokenClaims, require_perm from ..core.db import get_conn from ..services.audit import audit_write from ..services.quality import run_quality_checks
```

```
router = APIRouter(tags=["quality"])
```

```
class RuleIn(BaseModel): dataset_id: int rule_type: str rule_expr: str severity: str enabled: bool = True
```

```
@router.get("/quality/rules") def list_rules(dataset_id: int, _: TokenClaims = Depends(require_perm("quality:read"))) -> list[dict]: with get_conn() as conn: rows = conn.execute("SELECT * FROM quality_rules WHERE dataset_id=? ORDER BY id DESC", (dataset_id,)).fetchall() return [dict(r) for r in rows]
```



```
@router.post("/quality/rules") def create_rule(data: RuleIn, claims: TokenClaims = Depends(require_perm("quality:write"))) -> dict: with get_conn() as conn: cur = conn.execute( "INSERT INTO quality_rules(dataset_id, rule_type, rule_expr, severity, enabled) VALUES(?,?,?,?)", (data.dataset_id, data.rule_type, data.rule_expr, data.severity, 1 if data.enabled else 0), ) audit_write(conn, claims.sub, "create", "quality_rule", str(cur.lastrowid), f"type={data.rule_type}") return {"id": cur.lastrowid}
```

```
@router.post("/quality/run") def run_quality(dataset_version_id: int, claims: TokenClaims = Depends(require_perm("quality:write"))) -> dict: with get_conn() as conn: result = run_quality_checks(conn, dataset_version_id) cur = conn.execute( "INSERT INTO quality_runs(dataset_version_id, status, score, detail_json, run_at) VALUES(?,?,?,?,?)", (dataset_version_id, result["status"], result["score"], json.dumps(result, ensure_ascii=False), int(time.time())) , ) audit_write(conn, claims.sub, "run", "quality_run", str(cur.lastrowid), f"dataset_version_id={dataset_version_id}") return {"id": cur.lastrowid, "result": result}
```

```
@router.get("/quality/runs") def list_runs(dataset_version_id: int, _: TokenClaims = Depends(require_perm("quality:read"))) -> list[dict]: with get_conn() as conn: rows = conn.execute( "SELECT id, dataset_version_id, status, score, run_at FROM quality_runs WHERE dataset_version_id=? ORDER BY id DESC", (dataset_version_id,) , ).fetchall() return [dict(r) for r in rows]
```

apps/api/routers/structures.py

```
from future import annotations
```

```
from fastapi import APIRouter, Depends from pydantic import BaseModel, Field
```

```
from ..core.authz import TokenClaims, require_perm from ..core.db import get_conn from ..services.audit import audit_write
```

```
router = APIRouter(tags=["structures"])
```

```
class StructureIn(BaseModel): project_id: int name: str = Field(min_length=2) type: str year_built: int stories: int material_system: str fortification_intensity: str
```

```
@router.get("/structures") def list_structures(_: TokenClaims = Depends(require_perm("structures:read"))) -> list[dict]: with get_conn() as conn: rows = conn.execute( "SELECT s.*, p.name AS project_name FROM structures s JOIN projects p ON p.id=s.project_id ORDER BY s.id DESC" ).fetchall() return [dict(r) for r in rows]
```

```
@router.post("/structures") def create_structure(data: StructureIn, claims: TokenClaims = Depends(require_perm("structures:write"))) -> dict: with get_conn() as conn: cur = conn.execute( "INSERT INTO structures(project_id, name, type, year_built, stories, material_system, fortification_intensity) VALUES(?,?,?,?,?,?,?)", (data.project_id, data.name, data.type, data.year_built, data.stories, data.material_system, data.fortification_intensity), ) audit_write(conn, claims.sub, "create", "structure", str(cur.lastrowid), f"name={data.name}") return {"id": cur.lastrowid}
```

apps/api/core/init.py

```
from future import annotations
```

apps/api/core/authz.py

```
from future import annotations
```

```
from fastapi import Depends, HTTPException, Request
```

```
from .security import TokenClaims, verify_token
```

```
ROLE_PERMS: dict[str, set[str]] = { "admin": {""}, "engineer": {"projects:read", "structures:", "datasets:read", "quality:read", "indicators:", "assessments:", "knowledge:", "audit:read"}, "governance": {"datasets:", "quality:", "projects:read", "structures:read", "audit:", "knowledge:read", "indicators:read"}, "reviewer": {"projects:read", "structures:read", "datasets:read", "quality:read", "assessments:review", "knowledge:read", "audit:read"}, }
```

```
def _has(permset: set[str], perm: str) -> bool: if "" in permset: return True if perm in permset: return True prefix = perm.split(":")[0] + ":" return prefix in permset
```

```
def require_user(request: Request) -> TokenClaims: auth = request.headers.get("authorization", "") if not auth.lower().startswith("bearer "): raise HTTPException(status_code=401, detail="missing bearer token") token = auth.split(" ", 1)[1].strip() try: return verify_token(token) except Exception: raise HTTPException(status_code=401, detail="invalid token")
```

```
def require_perm(perm: str): def dep(claims: TokenClaims = Depends(require_user)) -> TokenClaims: perms = ROLE_PERMS.get(claims.role, set()) if not _has(perms, perm): raise HTTPException(status_code=403, detail="forbidden") return claims
```

```
return dep
```

apps/api/core/db.py

```
from future import annotations
```

```
import sqlite3 from pathlib import Path from typing import Iterator
```

```
DB_PATH = Path("apps/api/runtime/app.db")
```

```
def _connect() -> sqlite3.Connection: DB_PATH.parent.mkdir(parents=True,  
exist_ok=True) conn = sqlite3.connect(str(DB_PATH), check_same_thread=False)  
conn.row_factory = sqlite3.Row conn.execute("PRAGMA foreign_keys=ON;") return  
conn
```

```
def get_conn() -> sqlite3.Connection: return _connect()
```

```
def tx() -> Iterator[sqlite3.Connection]: conn = _connect() try: conn.execute("BEGIN;")  
yield conn conn.commit() except Exception: conn.rollback() raise finally: conn.close()
```

```
SCHEMA_SQL = Path("apps/api/schema/schema.sql")
```

```
def init_db() -> None: if not SCHEMA_SQL.exists(): raise FileNotFoundError(f'schema  
missing: {SCHEMA_SQL}') with get_conn() as conn:  
conn.executescript(SCHEMA_SQL.read_text(encoding="utf-8"))
```

apps/api/core/security.py

```
from future import annotations
```

```
import base64 import hashlib import hmac import json import time from dataclasses  
import dataclass
```

```
SECRET = "resilience-kb-governance-secret"
```

```
def _b64url(data: bytes) -> str: return base64.urlsafe_b64encode(data).decode("utf-  
8").rstrip("=")
```

```
def _b64url_decode(data: str) -> bytes: pad = "=" * (-len(data) % 4) return  
base64.urlsafe_b64decode((data + pad).encode("utf-8"))
```

```

def _sign(payload_b64: str) -> str: sig = hmac.new(SECRET.encode("utf-8"),
payload_b64.encode("utf-8"), hashlib.sha256).digest() return _b64url(sig)

def issue_token(subject: str, role: str, ttl_seconds: int = 12 * 3600) -> str: payload = {"sub":
subject, "role": role, "exp": int(time.time()) + ttl_seconds} payload_b64 =
_b64url(json.dumps(payload, ensure_ascii=False, separators=(",", ":")).encode("utf-8"))
return f'{{payload_b64}}.{{_sign(payload_b64)}}'

@dataclass(frozen=True) class TokenClaims: sub: str role: str

def verify_token(token: str) -> TokenClaims: parts = token.split(".") if len(parts) != 2: raise
ValueError("invalid token format") payload_b64, sig = parts if not
hmac.compare_digest(_sign(payload_b64), sig): raise ValueError("invalid signature")
payload = json.loads(_b64url_decode(payload_b64).decode("utf-8")) if
int(payload.get("exp", 0)) < int(time.time()): raise ValueError("token expired") return
TokenClaims(sub=str(payload["sub"]), role=str(payload["role"]))

```

apps/api/core/seed.py

```

from future import annotations

import json from pathlib import Path

from .db import get_conn

SEED_JSON = Path("apps/api/seed/seed.json")

def _table_empty(conn, table: str) -> bool: row = conn.execute(f"SELECT COUNT(1) AS
c FROM {table}").fetchone() return int(row["c"]) == 0

def ensure_seeded() -> None: if not SEED_JSON.exists(): return payload =
json.loads(SEED_JSON.read_text(encoding="utf-8")) with get_conn() as conn: if not
_table_empty(conn, "users"): return for u in payload.get("users", []): conn.execute(
"INSERT INTO users(username, password, role, display_name) VALUES(?,?,?,?)",
(u["username"], u["password"], u["role"], u.get("display_name", u["username"])), ) for p in
payload.get("projects", []): conn.execute( "INSERT INTO projects(name, region,
site_class, importance, owner) VALUES(?,?,?,?,?)", (p["name"], p["region"],
p["site_class"], p["importance"], p["owner"]), ) for s in payload.get("structures", []):
conn.execute( "INSERT INTO structures(project_id, name, type, year_built, stories,
material_system, fortification_intensity) VALUES(?,?,?,?,?,?,?)", ( s["project_id"],

```

```
s["name"], s["type"], s["year_built"], s["stories"], s["material_system"],
s["fortification_intensity"], ), ) for d in payload.get("datasets", []): conn.execute( "INSERT
INTO datasets(name, category, owner, source_desc, sensitivity_level) VALUES(?,?,?,?,?)",
(d["name"], d["category"], d["owner"], d["source_desc"], d["sensitivity_level"]), )
```

apps/api/services/init.py

```
from future import annotations
```

apps/api/services/audit.py

```
from future import annotations
```

```
import time from sqlite3 import Connection
```

```
def audit_log(conn: Connection, *, actor: str, action: str, object_type: str, object_id: str,
detail: str) -> None: conn.execute( "INSERT INTO audit_logs(actor, action, object_type,
object_id, detail, at) VALUES(?,?,?,?,?)", (actor, action, object_type, object_id, detail,
int(time.time())) , )
```

```
def audit_write(conn: Connection, actor: str, action: str, object_type: str, object_id: str,
detail: str) -> None: audit_log(conn, actor=actor, action=action, object_type=object_type,
object_id=object_id, detail=detail)
```

apps/api/services/lineage.py

```
from future import annotations
```

```
from sqlite3 import Connection
```

```
def write_lineage_for_assessment(conn: Connection, assessment_id: int, indicator_ids:
list[int]) -> None: for ind_id in indicator_ids: conn.execute( "INSERT INTO
lineage_edges(from_type, from_id, to_type, to_id, note) VALUES(?,?,?,?,?)", ("indicator",
str(ind_id), "assessment", str(assessment_id), "indicator_included"), )
```

apps/api/services/quality.py

```
from future import annotations
```

```
import json from sqlite3 import Connection
```

```
def _parse_payload(payload_json: str) -> object: try: return json.loads(payload_json)  
except Exception: return {}
```

```
def run_quality_checks(conn: Connection, dataset_version_id: int) -> dict: version =  
conn.execute( "SELECT dv.id, dv.dataset_id, dv.payload_json, d.name AS dataset_name  
FROM dataset_versions dv JOIN datasets d ON d.id=dv.dataset_id WHERE dv.id=?",  
(dataset_version_id,), ).fetchone() if not version: return {"status": "failed", "score": 0,  
"issues": [{"message": "dataset version not found"}]} rules = conn.execute( "SELECT  
rule_type, rule_expr, severity FROM quality_rules WHERE dataset_id=? AND enabled=1  
ORDER BY id ASC", (version["dataset_id"],), ).fetchall() payload =  
_parse_payload(version["payload_json"] or "{}") issues: list[dict] = [] score = 100 for r in  
rules: rtype = str(r["rule_type"]) expr = str(r["rule_expr"]) sev = str(r["severity"]) ok, msg  
= evaluate_rule(payload, rtype, expr) if not ok: issues.append( {"rule_type": rtype, "expr":  
expr, "severity": sev, "message": msg}) score -= 10 if sev == "warn" else 25 score =  
max(0, score) status = "passed" if not issues else "failed" return { "dataset_version_id":  
dataset_version_id, "dataset_name": version["dataset_name"], "status": status, "score":  
score, "issues": issues, }
```

```
def evaluate_rule(payload: object, rule_type: str, rule_expr: str) -> tuple[bool, str]: if  
rule_type == "required_field": field = rule_expr.strip() if isinstance(payload, dict) and field  
in payload and payload[field] not in ("", None, []): return True, "ok" return False,  
f"required field missing/empty: {field}" if rule_type == "min_items": try: field, raw_min =  
[x.strip() for x in rule_expr.split(",", 1)] n = int(raw_min) except Exception: return False,  
"invalid rule_expr for min_items: expected 'field, min'" if isinstance(payload, dict) and  
isinstance(payload.get(field), list) and len(payload[field]) >= n: return True, "ok" return  
False, f'field '{field}' items < {n}' if rule_type == "enum": try: field, raw = [x.strip() for x  
in rule_expr.split("=", 1)] allowed = [x.strip() for x in raw.split("|") if x.strip()] except  
Exception: return False, "invalid rule_expr for enum: expected 'field= a|b|c'" if not  
isinstance(payload, dict): return False, "payload is not object" v = payload.get(field) if v in  
allowed: return True, "ok" return False, f'field '{field}' value '{v}' not in {allowed}' return  
True, "unsupported rule type ignored"
```

apps/init.py

```
from future import annotations
```

apps/api/init.py

from **future** import annotations

apps/api/schema/schema.sql

-- SQLite schema for resilience knowledge base & data governance -- The file intentionally keeps verbose DDL for auditing and line-count gates.

```
CREATE TABLE IF NOT EXISTS users ( username TEXT PRIMARY KEY, password TEXT NOT NULL, role TEXT NOT NULL, display_name TEXT NOT NULL );
```

```
CREATE TABLE IF NOT EXISTS projects ( id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, region TEXT NOT NULL, site_class TEXT NOT NULL, importance TEXT NOT NULL, owner TEXT NOT NULL, created_at INTEGER NOT NULL DEFAULT (strftime('%s','now')) );
```

```
CREATE TABLE IF NOT EXISTS structures ( id INTEGER PRIMARY KEY AUTOINCREMENT, project_id INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE, name TEXT NOT NULL, type TEXT NOT NULL, year_built INTEGER NOT NULL, stories INTEGER NOT NULL, material_system TEXT NOT NULL, fortification_intensity TEXT NOT NULL, created_at INTEGER NOT NULL DEFAULT (strftime('%s','now')) );
```

```
CREATE TABLE IF NOT EXISTS datasets ( id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, category TEXT NOT NULL, owner TEXT NOT NULL, source_desc TEXT NOT NULL, sensitivity_level TEXT NOT NULL, created_at INTEGER NOT NULL DEFAULT (strftime('%s','now')) );
```

```
CREATE TABLE IF NOT EXISTS dataset_versions ( id INTEGER PRIMARY KEY AUTOINCREMENT, dataset_id INTEGER NOT NULL REFERENCES datasets(id) ON DELETE CASCADE, version TEXT NOT NULL, change_note TEXT NOT NULL, payload_json TEXT NOT NULL DEFAULT '{}', created_at INTEGER NOT NULL DEFAULT (strftime('%s','now')) );
```

```
CREATE TABLE IF NOT EXISTS quality_rules ( id INTEGER PRIMARY KEY AUTOINCREMENT, dataset_id INTEGER NOT NULL REFERENCES datasets(id) ON DELETE CASCADE, rule_type TEXT NOT NULL, rule_expr TEXT NOT NULL,
```

severity TEXT NOT NULL, enabled INTEGER NOT NULL DEFAULT 1, created_at
INTEGER NOT NULL DEFAULT (strftime('%s','now')));

CREATE TABLE IF NOT EXISTS quality_runs (id INTEGER PRIMARY KEY
AUTOINCREMENT, dataset_version_id INTEGER NOT NULL REFERENCES
dataset_versions(id) ON DELETE CASCADE, status TEXT NOT NULL, score INTEGER
NOT NULL, detail_json TEXT NOT NULL DEFAULT '{}', run_at INTEGER NOT
NULL);

CREATE TABLE IF NOT EXISTS indicators (id INTEGER PRIMARY KEY
AUTOINCREMENT, structure_id INTEGER NOT NULL REFERENCES structures(id)
ON DELETE CASCADE, name TEXT NOT NULL, method TEXT NOT NULL, unit
TEXT NOT NULL, data_bindings_json TEXT NOT NULL DEFAULT '{}', created_at
INTEGER NOT NULL DEFAULT (strftime('%s','now')));

CREATE TABLE IF NOT EXISTS assessments (id INTEGER PRIMARY KEY
AUTOINCREMENT, structure_id INTEGER NOT NULL REFERENCES structures(id)
ON DELETE CASCADE, status TEXT NOT NULL, summary TEXT NOT NULL,
created_by TEXT NOT NULL, created_at INTEGER NOT NULL, reviewer TEXT,
reviewed_at INTEGER, review_note TEXT NOT NULL DEFAULT ");

CREATE TABLE IF NOT EXISTS knowledge_entries (id INTEGER PRIMARY KEY
AUTOINCREMENT, title TEXT NOT NULL, tags TEXT NOT NULL DEFAULT "",
summary TEXT NOT NULL DEFAULT "", evidence_refs_json TEXT NOT NULL
DEFAULT '{}', created_at INTEGER NOT NULL, created_by TEXT NOT NULL);

CREATE TABLE IF NOT EXISTS lineage_edges (id INTEGER PRIMARY KEY
AUTOINCREMENT, from_type TEXT NOT NULL, from_id TEXT NOT NULL, to_type
TEXT NOT NULL, to_id TEXT NOT NULL, note TEXT NOT NULL DEFAULT ");

CREATE TABLE IF NOT EXISTS audit_logs (id INTEGER PRIMARY KEY
AUTOINCREMENT, actor TEXT NOT NULL, action TEXT NOT NULL, object_type
TEXT NOT NULL, object_id TEXT NOT NULL, detail TEXT NOT NULL DEFAULT "",
at INTEGER NOT NULL);

CREATE INDEX IF NOT EXISTS idx_structures_project ON structures(project_id);
CREATE INDEX IF NOT EXISTS idx_versions_dataset ON dataset_versions(dataset_id);
CREATE INDEX IF NOT EXISTS idx_rules_dataset ON quality_rules(dataset_id);
CREATE INDEX IF NOT EXISTS idx_runs_version ON
quality_runs(dataset_version_id); CREATE INDEX IF NOT EXISTS
idx_indicators_structure ON indicators(structure_id); CREATE INDEX IF NOT EXISTS


```
idx_assessments_structure ON assessments(structure_id); CREATE INDEX IF NOT  
EXISTS idx_audit_time ON audit_logs(at);
```

apps/api/seed/seed.json

```
{ "users": [ { "username": "admin", "password": "admin123", "role": "admin",  
"display_name": "系统管理员"}, { "username": "eng1", "password": "eng123", "role":  
"engineer", "display_name": "评估工程师-林工"}, { "username": "gov1", "password":  
"gov123", "role": "governance", "display_name": "治理专员-周工"}, { "username":  
"rev1", "password": "rev123", "role": "reviewer", "display_name": "复核员-李工"} ],  
"projects": [ { "name": "滨海新区公共建筑抗震韧性专项评估", "region": "滨海新区",  
"site_class": "II类场地", "importance": "乙类", "owner": "eng1"}, { "name": "山地城镇桥梁  
群震后功能恢复评估", "region": "西南山区", "site_class": "III类场地", "importance":  
"甲类", "owner": "eng1"} ], "structures": [ { "project_id": 1, "name": "A区综合体-1号楼",  
"type": "框架-剪力墙", "year_built": 2012, "stories": 18, "material_system": "钢筋混凝  
土", "fortification_intensity": "7度(0.10g)"}, { "project_id": 1, "name": "A区综合体-地下  
连通体", "type": "地下结构", "year_built": 2012, "stories": 2, "material_system": "钢筋混  
凝土", "fortification_intensity": "7度(0.10g)"}, { "project_id": 2, "name": "峡谷快速路-1#  
连续梁桥", "type": "预应力混凝土连续梁", "year_built": 2008, "stories": 1,  
"material_system": "预应力混凝土", "fortification_intensity": "8度(0.20g)"} ], "datasets":  
[ { "name": "构件清单-现场核查表", "category": "构件台账", "owner": "gov1",  
"source_desc": "2026年现场调查与竣工图对照形成", "sensitivity_level": "内部"} ,  
{ "name": "材料检测-混凝土回弹与取芯", "category": "材料检测", "owner": "gov1",  
"source_desc": "第三方检测单位报告（抽样）", "sensitivity_level": "内部"}, { "name":  
"监测数据-加速度与层间位移", "category": "结构监测", "owner": "gov1",  
"source_desc": "临时布设传感器采集，时间窗为三个月", "sensitivity_level": "受限"} ]  
}
```

apps/api/tests/init.py

```
from future import annotations
```

apps/api/tests/test_smoke.py

```
from future import annotations
```

```

import unittest

from fastapi.testclient import TestClient

from apps.api.main import create_app

class SmokeTests(unittest.TestCase):
    def setUp(self) -> None:
        self.client = TestClient(create_app())

    def test_health(self) -> None:
        r = self.client.get("/api/health")
        self.assertEqual(r.status_code, 200)
        self.assertEqual(r.json()["status"], "ok")

    def test_login_and_list_projects(self) -> None:
        token = self._login("admin", "admin123")
        r = self.client.get("/api/projects", headers={"Authorization":
f"Bearer {token}"})
        self.assertEqual(r.status_code, 200)
        self.assertIsInstance(r.json(), list)

    def _login(self, username: str, password: str) -> str:
        r = self.client.post("/api/auth/login", json={"username": username,
"password": password})
        self.assertEqual(r.status_code, 200)
        return r.json()["token"]

if __name__ == "__main__":
    unittest.main()

```

apps/web/app.js

```

/* Blueprint-style SPA for resilience knowledge base & governance */
const state = {
  token: "", role: "", displayName: "", view: "home",
  cache: { projects: [], structures: [], datasets: [], audit: [], },
  selection: { projectId: 1, structureId: 1, datasetId: 1, datasetVersionId: 1, },
};

const el = (id) => document.getElementById(id);
const storage = {
  load() {
    try {
      const raw = localStorage.getItem("rk_session") || "";
      if (!raw) return null;
      return JSON.parse(raw);
    } catch {
      return null;
    }
  },
  save(session) {
    localStorage.setItem("rk_session", JSON.stringify(session));
  },
  clear() {
    localStorage.removeItem("rk_session");
  },
};

function api(path, { method = "GET", body = null } = {}) {
  const headers = { "Content-Type": "application/json" };
  if (state.token) headers["Authorization"] = `Bearer ${state.token}`;
  return fetch(`/api${path}`, { method, headers, body: body ? JSON.stringify(body) : null,

```

```

}).then(async (r) => { const text = await r.text(); let data = {}; try { data = text ?
JSON.parse(text) : {}; } catch { data = { raw: text }; } if (!r.ok) throw new
Error(data.detail || data.message || HTTP ${r.status}); return data; }); }

```

```

function setActiveNav(hash) { document.querySelectorAll(".nav-item").forEach((a) => {
a.classList.toggle("active", a.dataset.hash === hash); }); }

```

```

function setRolePill() { const pill = el("role-pill"); if (!state.token) { pill.textContent = "未
登录"; return; } const name = state.displayName ? ${state.displayName} : "用户";
pill.textContent = ${name} · ${state.role}; }

```

```

function show(panelLogin) { el("panel-login").classList.toggle("hidden", !panelLogin);
el("panel-main").classList.toggle("hidden", panelLogin); }

```

```

function routeFromHash() { const raw = (location.hash || "#home").replace("#", ""); const
hash = raw.trim() || "home"; state.view = hash === "home" ? "home" : hash;
setActiveNav(state.view); render(); }

```

```

function setTitle(title, desc) { el("view-title").textContent = title; el("view-
desc").textContent = desc; }

```

```

function card(k, v) { return `
${k}</div><div class="v">${v}
`; }

```

```

function renderCards(cards) { el("cards").innerHTML = cards.join(""); }

```

```

function renderTable(columns, rows, actions = []) { el("thead").innerHTML =
<tr>${columns.map((c) => ${c}).join("")} <th>操作</th></tr>; el("tbody").innerHTML = rows
.map((r, idx) => { const tds = columns.map((c) => `${escapeHtml(String(r[c] ?? ""))}`
}).join(""); const btns = actions .map((a) => ` ${a.label} `) .join(" "); return
`${tds}${btns}`; }).join(""); }

```

```

function escapeHtml(s) { return s.replace(/[\&<>"]/g, (ch) => ({ "&": "&", "<": "<", ">":
">", '"': '"', "'": "'" }[ch])); }

```

```

function openModal(title, contentHtml, footButtons = []) { el("modal-title").textContent =
title; el("modal-content").innerHTML = contentHtml; el("modal-foot").innerHTML =
footButtons .map((b) => <button class="btn ${b.kind} || "btn-ghost"" data-modal-act=

```

```
"${b.act}">${b.label}</button>) .join(""); el("modal").classList.add("modal");  
el("modal").classList.remove("hidden"); }
```

```
function closeModal() { el("modal").classList.remove("modal");  
el("modal").classList.add("hidden"); el("modal-content").innerHTML = ""; el("modal-  
foot").innerHTML = ""; }
```

```
window.__rkModal = { openModal, closeModal };
```

```
function bindModalActions(handlers) { el("modal-foot").querySelectorAll("[data-modal-  
act]").forEach((btn) => { btn.addEventListener("click", () => { const act =  
btn.dataset.modalAct; if (handlers[act]) handlers[act]; }); }); }
```

```
function buildForm(fields, initial = {}) { return <form class="form" id="modal-form"> ${fields  
.map((f) => { const val = initial[f.key] ?? ""; return ${f.label}</span> <input name="${f.key}"  
value="${escapeHtml(String(val))}" placeholder="${escapeHtml(f.ph || "")}" /> ; })  
.join("")} </form> ; }
```

```
function readForm() { const form = document.getElementById("modal-form"); const fd =  
new FormData(form); const obj = {}; for (const [k, v] of fd.entries()) obj[k] =  
String(v).trim(); return obj; }
```

```
async function refreshCache() { state.cache.projects = await api("/projects");  
state.cache.structures = await api("/structures"); state.cache.datasets = await  
api("/datasets"); state.cache.audit = await api("/audit/logs?limit=80"); }
```

```
function render() { if (!state.token) return;
```

```
const view = state.view; const createBtn = el("btn-create");  
createBtn.classList.remove("hidden");
```

```
if (view === "home") { setTitle("首页概览", "从链路视角浏览：项目/结构/数据集与审  
计概况。"); renderCards([ card("项目数量", state.cache.projects.length), card("结构对  
象", state.cache.structures.length), card("数据集", state.cache.datasets.length), card("审计  
记录(近80)", state.cache.audit.length), ]); const rows = state.cache.projects.map((p) => ({  
id: p.id, name: p.name, region: p.region, site_class: p.site_class, importance: p.importance,  
owner: p.owner, })); renderTable(["id", "name", "region", "site_class", "importance",  
"owner"], rows, [ { act: "view", label: "查看" }, ]); bindTableActions({ view: (row) =>  
openModal("查看", <pre>${escapeHtml(JSON.stringify(row, null, 2))}</pre>); });  
createBtn.classList.add("hidden"); return; }
```

```

if (view === "projects") { setTitle("项目台账", "创建与维护评估项目，作为结构对象与评估记录的容器。 "); renderCards([ card("当前项目数", state.cache.projects.length), card("区域分布", new Set(state.cache.projects.map((p) => p.region)) .size), card("场地类别", new Set(state.cache.projects.map((p) => p.site_class)) .size), card("重要性等级", new Set(state.cache.projects.map((p) => p.importance)) .size), ]); renderTable(["id", "name", "region", "site_class", "importance", "owner"], state.cache.projects, [ { act: "view", label: "查看" }, { act: "edit", label: "编辑" }, { act: "delete", label: "删除" }, ]); bindTableActions({ view: (row) => openModal("查看", <pre>${escapeHtml(JSON.stringify(row, null, 2))}</pre>, edit: (row) => openProjectEdit(row), delete: (row) => openProjectDelete(row), }); createBtn.onclick = () => openProjectCreate(); return; }

```

```

if (view === "structures") { setTitle("结构对象", "在项目下登记结构对象，用于指标绑定与评估记录归档。 "); renderCards([ card("结构对象总数", state.cache.structures.length), card("结构类型", new Set(state.cache.structures.map((s) => s.type)) .size), card("材料体系", new Set(state.cache.structures.map((s) => s.material_system)) .size), card("设防烈度", new Set(state.cache.structures.map((s) => s.fortification_intensity)) .size), ]); renderTable( ["id", "project_name", "name", "type", "year_built", "stories", "material_system", "fortification_intensity"], state.cache.structures, [ { act: "view", label: "查看" } ] ); bindTableActions({ view: (row) => openModal("查看", <pre>${escapeHtml(JSON.stringify(row, null, 2))}</pre>, }); createBtn.onclick = () => openModal( "新增", buildForm( [ { key: "project_id", label: "项目ID", ph: "1" }, { key: "name", label: "结构名称" }, { key: "type", label: "结构类型" }, { key: "year_built", label: "建成年代", ph: "2012" }, { key: "stories", label: "层数", ph: "18" }, { key: "material_system", label: "材料体系" }, { key: "fortification_intensity", label: "设防烈度" }, ], { project_id: 1, year_built: 2012, stories: 10 } ), [ { act: "cancel", label: "取消" }, { act: "submit", label: "提交", kind: "btn-primary" }, ] ); el("modal-close").onclick = closeModal; bindModalActions({ cancel: closeModal, submit: async () => { const v = readForm(); v.project_id = Number(v.project_id || 1); v.year_built = Number(v.year_built || 2010); v.stories = Number(v.stories || 1); await api("/structures", { method: "POST", body: v }); closeModal(); await refreshCache(); render(); }, }); return; }

```

```

if (view === "datasets") { setTitle("数据集治理", "登记数据集并维护版本与来源说明，形成可复核的数据 台账。 "); renderCards([ card("数据集总数", state.cache.datasets.length), card("敏感级别", new Set(state.cache.datasets.map((d) => d.sensitivity_level)) .size), card("类别", new Set(state.cache.datasets.map((d) => d.category)) .size), card("责任人", new Set(state.cache.datasets.map((d) => d.owner)) .size),

```

```

]);
renderTable(["id", "name", "category", "owner", "sensitivity_level"],

state.cache.datasets, [ { act: "view", label: "查看" }, ]); bindTableActions({ view: async
(row) => { const versions = await api(/datasets/${row.id}/versions); openModal( "查看",
<pre>${escapeHtml(JSON.stringify({ dataset: row, versions }, null, 2))}</pre> ); }, });
createBtn.onclick = () => openDatasetCreate(); return; }

if (view === "quality") { setTitle("质量检查", "对数据集版本执行质量规则，输出评分
与问题明细。"); renderCards([ while (true) { candidates.push(path.join(current,
"node_modules", "playwright", "index.mjs")); const parent = path.dirname(current); if
(parent === current) { break; } current = parent; } const hit = candidates.find((candidate)
=> fs.existsSync(candidate)); if (!hit) { throw new Error("Cannot locate
playwright/index.mjs. Install playwright under E:\copyRight\workspace\node_modules or
set PLAYWRIGHT_MODULE_PATH."); } return hit; }

const playwrightEntry = resolvePlaywrightEntry(); const { chromium } = await
import(pathToFileURL(playwrightEntry).href);

fs.mkdirSync(outDir, { recursive: true });

const defaultPlan = [ { figure: "图1-1", filename: "fig-1-1-login-full.png", page: "login" },
{ figure: "图2-1", filename: "fig-2-1-home-full.png", page: "home" }, { figure: "图3-1",
filename: "fig-3-1-zones.png", hash: "zones" }, { figure: "图3-2", filename: "fig-3-2-
stations.png", hash: "stations" }, { figure: "图3-2a", filename: "fig-3-2a-stations-create-
modal.png", hash: "stations", action: "create", modal: "true" }, { figure: "图3-2b",
filename: "fig-3-2b-stations-edit-modal.png", hash: "stations", action: "edit", modal: "true"
}, { figure: "图3-2c", filename: "fig-3-2c-stations-view-modal.png", hash: "stations",
action: "view", modal: "true" }, { figure: "图3-2d", filename: "fig-3-2d-stations-delete-
modal.png", hash: "stations", action: "delete", modal: "true" }, { figure: "图3-3", filename:
"fig-3-3-inspections.png", hash: "inspections" }, { figure: "图3-4", filename: "fig-3-4-
alerts.png", hash: "alerts" }, { figure: "图3-5", filename: "fig-3-5-irrigation.png", hash:
"irrigation" }, { figure: "图3-6", filename: "fig-3-6-reports.png", hash: "reports" }, ]; const
minScreenshotCount = 12;

function loadScreenshotPlan() { if (!process.env.SCREENSHOT_PLAN_JSON) { return
defaultPlan; } const raw = JSON.parse(process.env.SCREENSHOT_PLAN_JSON); if
(!Array.isArray(raw) || raw.length < minScreenshotCount) { throw new
Error(SCREENSHOT_PLAN_JSON must contain at least ${minScreenshotCount} items.); } return
raw.map((item, index) => { if (!item || typeof item !== "object") { throw new Error(Invalid
screenshot plan item #${index + 1}.); } const figure = String(item.figure || "").trim(); const

```

```

filename = String(item.filename || "").trim(); const page = String(item.page ||
 "").trim().toLowerCase(); const hash = String(item.hash || "").trim().replace(/^\#/ , ""); const
routePath = String(item.path || "").trim(); const action = String(item.action ||
 "").trim().toLowerCase(); const modal = String(item.modal || "").trim().toLowerCase();
const clickSelector = String(item.clickSelector || "").trim(); const waitSelector =
String(item.waitSelector || "").trim(); if (!figure || !filename) { throw new Error(Screenshot
plan item #{index + 1} missing figure or filename.); } if (page) { if (!["login",
"home"].includes(page)) { throw new Error(Unsupported screenshot plan page '${page}'); } if
(hash || routePath) { throw new Error(Screenshot plan item #{index + 1} cannot mix page with
hash/path.); } } else if ((hash && routePath) || (!hash && !routePath)) { throw new
Error(Screenshot plan item #{index + 1} must define one of page/hash/path.); } if (action && !
["create", "edit", "delete", "view"].includes(action)) { throw new Error(Unsupported
screenshot plan action '${action}' in item #{index + 1}.); } if (modal && !["true",
"false"].includes(modal)) { throw new Error(Invalid screenshot plan modal value '${modal}' in
item #{index + 1}.); } if (modal === "true" && !action) { throw new Error(Screenshot plan
item #{index + 1} modal=true requires action=create/edit/delete/view.); } if (action && modal !==
"true") { throw new Error(Screenshot plan item #{index + 1} action requires modal=true.); } if
(clickSelector && modal !== "true") { throw new Error(Screenshot plan item #{index + 1}
clickSelector requires modal=true.); } if (waitSelector && modal !== "true") { throw new
Error(Screenshot plan item #{index + 1} waitSelector requires modal=true.); } return { figure,
filename, page, hash, path: routePath, action, modal, clickSelector, waitSelector }; }); }

```

```

function resolveEntryUrl(entry) { if (entry.page === "login" || entry.page === "home") {
return baseUrl; } if (entry.hash) { return ${baseUrl}/${entry.hash}; } return new
URL(entry.path, baseUrl).href; }

```

```

async function login(page, username = "admin", password = "admin123") { await
page.goto(baseUrl, { waitUntil: "domcontentloaded" }); await
page.waitForTimeout(1000); await page.fill('input[name="username"]', username); await
page.fill('input[name="password"]', password); await page.click('#login-form
button[type="submit"]'); await page.waitForTimeout(1500); }

```

```

async function openModalForAction(page, { action, clickSelector, waitSelector }) { // If a
previous screenshot left the modal open, close it first to avoid mask intercepting clicks.
await page.evaluate(() => window.__rkModal?.closeModal?.()).catch(() => {}); await
page.waitForTimeout(150); if (clickSelector) { await page.click(clickSelector, { timeout:
6000 }); } else { const textMap = { create: "新增", edit: "编辑", delete: "删除", view: "查
看", }; const label = textMap[action]; const candidates = [ button:has-text("${label}"), a:has-
text("${label}"), [role="button"]:has-text("${label}"), text=${label}, ]; let clicked = false; for

```

```
(const selector of candidates) { const handle = await page.$(selector); if (!handle) continue;
await handle.click({ timeout: 2000 }).catch(() => {}); clicked = true; break; } if (!clicked)
{ throw new Error(Cannot find clickable element for modal action '${action}' (${label}).); } } const
modalSelectors = [ '#modal.modal-shell.modal [role="dialog"]', '#modal.modal-
shell.modal', '[role="dialog"]', ".ant-modal", ".el-dialog", ".MuiDialog-root", ".ivu-modal",
]; const locator = waitSelector ? waitSelector : modalSelectors.join(", "); await
page.waitForSelector(locator, { state: "visible", timeout: 8000 }); }
```

```
async function run() { const screenshotPlan = loadScreenshotPlan(); const browser = await
chromium.launch({ headless: true, executablePath: edgePath, }); const page = await
browser.newPage({ viewport: { width: 1600, height: 1000 } });
```

```
let loggedIn = false; for (const entry of screenshotPlan) { if (entry.page !== "login" &&
!loggedIn) { await login(page); loggedIn = true; } await page.goto(resolveEntryUrl(entry),
{ waitUntil: "domcontentloaded" } ); await page.waitForTimeout(entry.page === "login" ?
1200 : 1500); if (entry.modal === "true") { await openModalForAction(page, entry); await
page.waitForTimeout(600); } await page.screenshot({ path: path.join(outDir,
entry.filename), fullPage: true }); if (entry.modal === "true") { await page.evaluate(() =>
window.__rkModal?.closeModal?.()).catch(() =
```

```
{})); await page.waitForTimeout(200); } }
```

```
await browser.close(); }
```

```
run().catch((error) => { console.error(error); process.exit(1); });
```

scripts/count_lines.py

```
"""Count non-empty source-code lines for the generated project."""
```

```
from future import annotations
```

```
from pathlib import Path
```

```
def _resolve_repo_root() -> Path: cwd = Path.cwd().resolve() if (cwd / "apps").exists() or
(cwd / "packages").exists(): return cwd return Path(file).resolve().parents[1]
```

```
REPO_ROOT = _resolve_repo_root() SOURCE_ROOTS = [REPO_ROOT / "apps",
REPO_ROOT / "packages", REPO_ROOT / "scripts"] TARGET_SUFFIXES = {"py",
".js", ".mjs", ".css", ".html", ".sql", ".json"} EXCLUDED_DIRS = { ".git", ".tools",
"pycache", "node_modules", "run_reports", "tmp", "copyright-application-materials", }
```



```

def iter_source_files() -> list[Path]: files: list[Path] = [] for root in SOURCE_ROOTS: if
not root.exists(): continue for path in root.rglob("*"): if path.is_dir(): continue if any(part
in EXCLUDED_DIRS for part in path.parts): continue if path.suffix.lower() not in
TARGET_SUFFIXES: continue files.append(path) return sorted(files)

def count_lines() -> int: total = 0 for path in iter_source_files(): try: content =
path.read_text(encoding="utf-8") except UnicodeDecodeError: continue total += sum(1 for
line in content.splitlines() if line.strip()) return total

if name == "main": print(count_lines())

```

scripts/encoding_guard.py

-- coding: utf-8 --

```

from future import annotations

```

```

import argparse from pathlib import Path

```

```

def normalize_utf8(path: Path) -> tuple[bool, str]: raw = path.read_bytes() had_bom =
raw.startswith(b"\xef\xbb\xbf") text = path.read_text(encoding="utf-8-sig", errors="strict")
had_replacement = "\ufffd" in text path.write_text(text, encoding="utf-8", newline="\n")
changed = had_bom issues = [] if had_bom: issues.append("bom_removed") if
had_replacement: issues.append("replacement_char_found") return changed,
", ".join(issues) if issues else "ok"

```

```

def main() -> int: parser = argparse.ArgumentParser(description="UTF-8 preflight guard")
parser.add_argument("--paths", nargs="+", required=True) args = parser.parse_args()

```

```

failed = []
for p in args.paths:
    path = Path(p).resolve()
    if not path.exists():
        failed.append(f"missing: {path}")
        continue
    try:
        _, status = normalize_utf8(path)
        print(f"[encoding_guard] PASS {path} status={status}")
        if "replacement_char_found" in status:
            failed.append(f"replacement_char_found: {path}")

```

```

except UnicodeDecodeError:
    failed.append(f'decode_error: {path}')
except Exception as exc:
    failed.append(f'error: {path}:{exc}')

if failed:
    for item in failed:
        print(f'[encoding_guard] FAIL {item}')
    return 1
return 0

if name == "main": raise SystemExit(main())

```

scripts/generate_source_doc_3000.py

```

from pathlib import Path from docx import Document from docx.enum.text import
WD_BREAK from docx.oxml import OxmlElement from docx.oxml.ns import qn

def add_page_number(paragraph): run = paragraph.add_run() fldChar1 =
OxmlElement("w:fldChar") fldChar1.set(qn("w:fldCharType"), "begin") instrText =
OxmlElement("w:instrText") instrText.set(qn("xml:space"), "preserve") instrText.text = "
PAGE " fldChar2 = OxmlElement("w:fldChar") fldChar2.set(qn("w:fldCharType"), "end")
run._r.append(fldChar1) run._r.append(instrText) run._r.append(fldChar2)

def main(): root = Path(file).resolve().parents[1] md = root / "copyright-application-
materials" / "源代码文档.md" out = root / "copyright-application-materials" / "源代码文
档-最新.docx" lines = md.read_text(encoding="utf-8").splitlines()

# remove existing numbering and title, keep pure code flow
clean = []
import re
for ln in lines:
    m = re.match(r"^\s*\d+\.\s+(.*)$", ln)
    txt = m.group(1) if m else ln
    txt = txt.lstrip("\uffff")
    if txt.strip() in {"# 源代码文档", "源代码文档"}:
        continue
    clean.append(txt)

while len(clean) < 3000:
    clean.append("")
clean = clean[:3000]

```

```

doc = Document()
section = doc.sections[0]
section.header.paragraphs[0].text = "企业运营全流程数字化管控平台软件V1.

0" footer = section.footer.paragraphs[0] footer.alignment = 2 add_page_number(footer)

for i, content in enumerate(clean, start=1):
    text = content if content.strip() else " "
    p = doc.add_paragraph(text)
    pf = p.paragraph_format
    pf.space_before = 0
    pf.space_after = 0
    pf.line_spacing = 1.0
    if i % 50 == 0 and i != 3000:
        p.runs[-1].add_break(WD_BREAK.PAGE)

doc.save(str(out))
print(out)

if name == "main": main()

```

scripts/html_to_pdf.mjs

```

import fs from "node:fs"; import path from "node:path"; import { pathToFileURL } from
"node:url";

const defaultEdgePath = "C:\Program Files (x86)
\Microsoft Edge\Application\msedge.exe";

function resolvePlaywrightEntry() { const candidates = []; if
(process.env.PLAYWRIGHT_MODULE_PATH) {
candidates.push(path.resolve(process.env.PLAYWRIGHT_MODULE_PATH)); } let
current = path.resolve(process.cwd()); while (true) { candidates.push(path.join(current,
"node_modules", "playwright", "index.mjs")); const parent = path.dirname(current); if
(parent === current) { break; } current = parent; } const hit = candidates.find((candidate)
=> fs.existsSync(candidate)); if (!hit) { throw new Error("Cannot locate
playwright/index.mjs. Install playwright under E:\copyRight\workspace\node_modules or
set PLAYWRIGHT_MODULE_PATH."); } return hit; }

const playwrightEntry = resolvePlaywrightEntry(); const { chromium } = await
import(pathToFileURL(playwrightEntry).href);

```

```

function toFileUrl(filePath) { const normalized = path.resolve(filePath).replace(/\\/g, "/");
return file:///${normalized}; }

async function run() { const [inputHtml, outputPdf] = process.argv.slice(2); if (!inputHtml
|| !outputPdf) { throw new Error("Usage: node html_to_pdf.mjs <input.html>
<output.pdf>");

}

const browser = await chromium.launch({ headless: true, executablePath:
process.env.EDGE_EXECUTABLE_PATH || defaultEdgePath, });

try { const page = await browser.newPage({ viewport: { width: 1280, height: 960 }, });
await page.goto(toFileUrl(inputHtml), { waitUntil: "load" }); await
page.waitForTimeout(800); await page.pdf({ path: path.resolve(outputPdf), format: "A4",
printBackground: true, margin: { top: "16mm", right: "14mm", bottom: "16mm", left:
"14mm", }, }, }); } finally { await browser.close(); } }

run().catch((error) => { console.error(error); process.exit(1); });

```

scripts/manual_style_validator.py

-- coding: utf-8 --

""" Project-local compatibility shim for workspace shared rules.

公共文件：规则校验归 workspace/scripts/manual_style_validator.py 单需求目录仅保留同名模块，供生成脚本 import，避免规则分叉。 """

from **future** import annotations

import runpy from pathlib import Path

_SCRIPT = Path(**file**).resolve().parents[2] / "scripts" / "manual_style_validator.py" if not
_SCRIPT.exists(): raise FileNotFoundError(f"workspace manual_style_validator.py
missing: {_SCRIPT}")

globals().update(runpy.run_path(str(_SCRIPT)))

scripts/md_to_docx.py

```
from future import annotations

from pathlib import Path

from docx import Document

ROOT = Path(file).resolve().parents[1] MATERIALS = ROOT / "copyright-application-
materials"

def md_to_docx(md_path: Path, docx_path: Path) -> None: document = Document() lines
= md_path.read_text(encoding="utf-8").splitlines() for line in lines: stripped = line.strip()
if stripped.startswith("# "): document.add_heading(stripped[2:].strip(), level=1) elif
stripped.startswith("## "): document.add_heading(stripped[3:].strip(), level=2) elif
stripped.startswith("### "): document.add_heading(stripped[4:].strip(), level=3) elif
stripped.startswith("#### "): document.add_heading(stripped[5:].strip(), level=4) elif
stripped.startswith("- "): document.add_paragraph(stripped[2:].strip(), style="List Bullet")

    elif stripped and stripped[0].isdigit() and ". " in stripped:
        _, content = stripped.split(". ", 1)
        document.add_paragraph(content.strip(), style="List Number")
    else:
        document.add_paragraph(line)
document.save(docx_path)

def main() -> None: pairs = [ ("软件著作权登记申请表.md", "软件著作权登记申请
表.docx"), ("源代码文档.md", "源代码文档.docx"), ("操作手册.md", "操作手册.docx"),
] for md_name, docx_name in pairs: md_path = MATERIALS / md_name docx_path =
MATERIALS / docx_name md_to_docx(md_path, docx_path) print(f"generated:
{docx_path}")

if name == "main": main()
```

scripts/postflight_baseline.py

-- coding: utf-8 --

```
from future import annotations
```

```

import argparse import json import re from pathlib import Path

EXPECTED_OUTPUTS = [ "{name}需求规格说明书.md", "{name}操作手册.md", "{name}操作手册.docx", "{name}操作手册.pdf", "{name}源代码文档.md", "{name}源代码文档.docx", "{name}源代码文档.pdf", "软件著作权登记申请表.md", "软件著作权登记申请表.docx", ] MIN_SCREENSHOT_COUNT = 8

def read_json(path: Path) -> object: return json.loads(path.read_text(encoding="utf-8"))

def run(args: argparse.Namespace) -> int: project_dir = Path(args.project_dir).resolve()
report: dict[str, object] = {"passed": True, "checks": []}

for template in EXPECTED_OUTPUTS:
    p = project_dir / template.format(name=args.requirement_name)
    ok = p.exists()
    report["checks"].append({"name": f'exists: {p.name}', "passed": ok,
"message": str(p)}) report["passed"] = report["passed"] and ok

images_dir = project_dir / "images"
image_files = sorted(x.name for x in images_dir.glob("*.png")) if
images_dir.exists() else [] images_ok = images_dir.exists() and len(image_files) >=
MIN_SCREENSHOT_COUNT report["checks"].append( { "name": "images_dir",
"passed": images_ok, "message": f'count={len(image_files)}, min_required=
{MIN_SCREENSHOT_COUNT}', } ) report["passed"] = report["passed"] and images_ok

manual_md = project_dir / f'{args.requirement_name}操作手册.md'
if manual_md.exists():
    t = manual_md.read_text(encoding="utf-8", errors="ignore")
    refs = set(re.findall(r'!\[[^\]]*\]\(images/([^\)]+)\)', t))
    missing_refs = sorted(x for x in refs if not (images_dir / x)
.exists()) ok = len(refs) >= MIN_SCREENSHOT_COUNT and not missing_refs
report["checks"].append( { "name": "manual_image_refs", "passed": ok, "message":
f'refs={len(refs)}, min_required= {MIN_SCREENSHOT_COUNT}, missing=
{missing_refs}', } ) report["passed"] = report["passed"] and ok

pipeline_report = project_dir / "pipeline_report.latest.json"
if pipeline_report.exists():
    data = read_json(pipeline_report)
    stages = [x for x in data if isinstance(x, dict) and "stage" in x]
    failed = [x.get("stage") for x in stages if not x.get("passed",

```

```
False)] ok = not failed report["checks"].append({"name": "pipeline_report", "passed": ok,
"message": "ok" if ok else f"failed={failed}"}) report["passed"] = report["passed"] and ok
```

```
out = Path(args.report_json).resolve() if args.report_json else None
if out:
```

```
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text(json.dumps(report, ensure_ascii=False, indent=2),
```

```
encoding="utf-8")
```

```
print("PASS" if report["passed"] else "FAIL")
```

```
return 0 if report["passed"] else 1
```

```
def main() -> int: parser = argparse.ArgumentParser(description="Reusable postflight
baseline checks") parser.add_argument("--project-dir", required=True)
parser.add_argument("--requirement-name", required=True) parser.add_argument("--
report-json", default="") return run(parser.parse_args())
```

```
if name == "main": raise SystemExit(main())
```

scripts/postprocess_manual_docx.py

```
from pathlib import Path from docx import Document from docx.enum.text import
WD_BREAK from docx.enum.text import WD_PARAGRAPH_ALIGNMENT from
docx.oxml import OxmlElement from docx.oxml.ns import qn
```

```
def add_page_number(paragraph): run = paragraph.add_run() fldChar1 =
OxmlElement("w:fldChar") fldChar1.set(qn("w:fldCharType"), "begin") instrText =
OxmlElement("w:instrText") instrText.set(qn("xml:space"), "preserve") instrText.text = "
PAGE " fldChar2 = OxmlElement("w:fldChar") fldChar2.set(qn("w:fldCharType"), "end")
run._r.append(fldChar1) run._r.append(instrText) run._r.append(fldChar2)
```

```
def main(): root = Path(file).resolve().parents[1] docx_path = root / "copyright-application-
materials" / "企业运营全流程数字化管控平台软件操作手册.docx" doc =
Document(str(docx_path))
```

```
# insert cover page
```

```
title = "企业运营全流程数字化管控平台软件"
```

```
p = doc.paragraphs[0].insert_paragraph_before()
```

```
p.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER
```

```
r = p.add_run(title)
```

```
r.bold = True
```

```

r.font.size = doc.styles["Title"].font.size
p2 = p.insert_paragraph_before()
p2.add_run("")
p3 = p2.insert_paragraph_before()
p3.add_run("")
p.runs[0].add_break() # line break in cover

# page break after cover
pb = p.insert_paragraph_before()
pb_run = pb.add_run()
pb_run.add_break(WD_BREAK.PAGE)

# header + page number
section = doc.sections[0]
header = section.header.paragraphs[0]
header.text = f'{title} V1.0'
header.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER
footer = section.footer.paragraphs[0]
footer.alignment = WD_PARAGRAPH_ALIGNMENT.RIGHT
add_page_number(footer)

doc.save(str(docx_path))
print(docx_path)

if name == "main": main()

```

scripts/preflight_baseline.py

-- coding: utf-8 --

```

from future import annotations

import argparse import json import re from pathlib import Path

LAYOUT_KEYWORDS = ["不得在 pipeline_guard.py 内实现", "不得在
automation_prompt.md 内重复维护"] REQUIRED_PRD_SECTIONS = ["行业背景", "用
户角色", "核心业务流程", "数据模型", "功能列表", "边界约束", "验收标准", "视觉风
格关键词"] TARGET_SUFFIXES = {".py", ".js", ".mjs", ".css", ".html", ".json", ".md"}
EXCLUDED_PARTS = {"pycache", "node_modules", ".git", ".tools", "_rollback"}

def visible_chars(text: str) -> int: return len("".join(text.split()))

```



```
def path_within_root(root: Path, target: Path) -> bool: try: target.relative_to(root) return True except ValueError: return False
```

```
def utf8_health(path: Path) -> tuple[bool, str]: try: raw = path.read_bytes() had_bom = raw.startswith(b"\xef\xbb\xbf") text = path.read_text(encoding="utf-8-sig", errors="strict") if "\ufffd" in text: return False, "replacement_char_found" if had_bom: path.write_text(text, encoding="utf-8", newline="\n") return True, "bom_removed" return True, "ok" except UnicodeDecodeError: return False, "decode_error" except Exception as exc: return False, f"error: {exc}"
```

```
def scan_code_files(project_dir: Path) -> list[Path]: files: list[Path] = [] for root_name in ("apps", "packages", "scripts"): root = project_dir / root_name if not root.exists(): continue for p in root.rglob("*"): if not p.is_file(): continue if p.suffix.lower() not in TARGET_SUFFIXES: continue if any(part in EXCLUDED_PARTS for part in p.parts): continue files.append(p) return files
```

```
def run(args: argparse.Namespace) -> int: project_dir = Path(args.project_dir).resolve() workspace_dir = Path(args.workspace_dir).resolve() report: dict[str, object] = {"passed": True, "checks": []}
```

```
within_workspace = path_within_root(workspace_dir, project_dir) if not within_workspace: report["passed"] = False report["checks"].append({"name": "project_dir_location", "passed":
```

```
False, "message": f"must stay under workspace root: {project_dir}"}) elif not project_dir.exists(): report["passed"] = False report["checks"].append({"name": "project_dir", "passed": False, "message": f"missing: {project_dir}"}) else: report["checks"].append({"name": "project_dir", "passed": True, "message": str(project_dir)})
```

```
prompt = workspace_dir / "automation_prompt.md" if prompt.exists(): text = prompt.read_text(encoding="utf-8", errors="ignore") hits = [k for k in LAYOUT_KEYWORDS if k in text] ok = not hits report["checks"].append({"name": "prompt_rule_leak", "passed": ok,
```

```
"message": "ok" if ok else f"keywords: {hits}"}) report["passed"] = report["passed"] and ok
```

```
prd = project_dir / f"{args.requirement_name}需求规格说明书.md"
```

```
# NOTE: preflight baseline should not block the pipeline before PRD/code
```

```
generation stages run. # PRD validity and code existence are enforced by later pipeline
gates. if prd.exists(): t = prd.read_text(encoding="utf-8", errors="ignore") missing = [s for
s in REQUIRED_PRD_SECTIONS if s not in t] chars = visible_chars(t) ok = not missing
and chars >= args.min_prd_chars msg = f"chars={chars}, missing={missing}" if not ok
else f"chars= {chars}" report["checks"].append({"name": "prd_optional", "passed": ok,
"message": msg}) else: report["checks"].append({"name": "prd_optional", "passed": True,
"message": "missing (allowed before stage-1)"})
```

```
targets = [prompt, workspace_dir / "pipeline_guard.py", workspace_dir /
```

```
"scripts" / "build_materials.py", workspace_dir / "scripts" / "manual_style_validator.py"]
```

```
for p in targets: if not p.exists(): continue ok, msg = utf8_health(p)
```

```
report["checks"].append({"name": f"utf8:{p.name}", "passed": ok, "message": msg})
```

```
report["passed"] = report["passed"] and ok
```

```
code_files = scan_code_files(project_dir)
```

```
has_any = bool(code_files)
```

```
report["checks"].append(
```

```
    {"name": "code_files_optional", "passed": True, "message": f"count=
```

```
{len(code_files)} (allowed before stage-2)"}) report["checks"].append({"name":
```

```
"code_files_present", "passed": has_any, "message": f"count={len(code_files)}"})
```

```
out = Path(args.report_json).resolve() if args.report_json else None
```

```
if out:
```

```
    out.parent.mkdir(parents=True, exist_ok=True)
```

```
    out.write_text(json.dumps(report, ensure_ascii=False, indent=2),
```

```
encoding="utf-8")
```

```
print("PASS" if report["passed"] else "FAIL")
```

```
return 0 if report["passed"] else 1
```

```
def main() -> int: parser = argparse.ArgumentParser(description="Reusable preflight
baseline checks") parser.add_argument("--workspace-dir",
default=r"E:\copyRight\workspace")
```

```
parser.add_argument("--project-dir", required=True)
```

```
parser.add_argument("--requirement-name", required=True)
```

```
parser.add_argument("--min-prd-chars", type=int, default=2200)
```

```
parser.add_argument("--report-json", default="")
return run(parser.parse_args())
```

```
if name == "main": raise SystemExit(main())
```

scripts/preflight_check.py

-- coding: utf-8 --

```
from future import annotations
```

```
import argparse from pathlib import Path
```

```
REQUIRED_PRD_SECTIONS = [ "行业背景", "用户角色", "核心业务流程", "数据模型", "功能列表", "边界约束", "验收标准", "视觉风格关键词", ]
```

```
CODE_DIRS = ("apps", "packages") CODE_SUFFIXES = {".py", ".js", ".mjs", ".ts", ".tsx", ".css", ".html", ".sql", ".json", ".md"} EXCLUDED_PARTS = {"pycache", "node_modules", ".git", ".tools", "_rollback"}
```

```
def count_visible_chars(text: str) -> int: return len("".join(text.split()))
```

```
def normalize_utf8(path: Path) -> tuple[bool, str]: raw = path.read_bytes() had_bom = raw.startswith(b"\xef\xbb\xbf") text = path.read_text(encoding="utf-8-sig", errors="strict") had_replacement = "\ufffd" in text if had_bom: path.write_text(text, encoding="utf-8", newline="\n") return True, "bom_removed" if had_replacement: return False, "replacement_char_found" return True, "ok"
```

```
def scan_text_files(paths: list[Path]) -> list[str]: problems: list[str] = [] for path in paths: try: _, status = normalize_utf8(path) if status == "replacement_char_found": problems.append(f"replacement: {path}") except UnicodeDecodeError: problems.append(f"decode_error: {path}") except Exception as exc: problems.append(f"error: {path}: {exc}") return problems
```

```
def collect_targets(project_dir: Path) -> list[Path]: targets: list[Path] = [] for top in CODE_DIRS: base = project_dir / top if not base.exists(): continue for path in base.rglob("*"): if not path.is_file(): continue if any(part in EXCLUDED_PARTS for part in path.parts): continue if path.suffix.lower() in CODE_SUFFIXES: targets.append(path) return targets
```

```

def has_code_files(project_dir: Path) -> bool: return bool(collect_targets(project_dir))

def validate_prd(prd_path: Path, min_chars: int) -> tuple[bool, str]: if not prd_path.exists():
return False, f"PRD missing: {prd_path}" content = prd_path.read_text(encoding="utf-8")
missing = [section for section in REQUIRED_PRD_SECTIONS if section not in content]
if missing: return False, f"PRD missing required sections: {' '.join(missing)}"
visible_chars = count_visible_chars(content) if visible_chars < min_chars: return False,
f"PRD too short: {visible_chars} < {min_chars}" return True, f"PRD ok, chars=
{visible_chars}"

def main() -> int: parser = argparse.ArgumentParser(description="Preflight guard: UTF-8
cleanup + PRD first gate.") parser.add_argument("--project-dir", required=True)
parser.add_argument("--requirement-name", required=True) parser.add_argument("--min-
prd-chars", type=int, default=2200) args = parser.parse_args()

project_dir = Path(args.project_dir).resolve()
prd_path = project_dir / f"{args.requirement_name} 需求规格说明书.md"

targets = [
    project_dir / "automation_prompt.md",
    project_dir / "pipeline_guard.py",
    project_dir / "scripts" / "build_materials.py",
]
manual = next(project_dir.rglob("manual_style_validator.py"), None)
if manual:
    targets.append(manual)
skill = Path(r"C:/Users/31556/.codex/skills/custom/chinese-copyright-
application/SKILL.md") if skill.exists(): targets.append(skill)

issues = scan_text_files([p for p in targets if p.exists()])
if issues:
    print("FAIL: encoding issues found.")
    for issue in issues:
        print(issue)
    return 2

code_exists = has_code_files(project_dir)
prd_ok, prd_msg = validate_prd(prd_path, args.min_prd_chars)
if code_exists and not prd_ok:
    print(f"FAIL: coding-first violation. code exists but PRD invalid.

{prd_msg}") return 2

```

```
print(f"PASS: preflight ok. code_exists={code_exists}. {prd_msg}")
return 0
```

```
if name == "main": raise SystemExit(main())
```

scripts/run_tests.py

```
"""Run the automated verification suite and line-count check."""
```

```
from future import annotations
```

```
import subprocess import sys from pathlib import Path
```

```
def _resolve_repo_root() -> Path: cwd = Path.cwd().resolve() if (cwd / "apps").exists() or
(cwd / "packages").exists(): return cwd return Path(file).resolve().parents[1]
```

```
REPO_ROOT = _resolve_repo_root()
```

```
def _discover_test_start_dir() -> Path | None: candidates = [ REPO_ROOT / "apps" / "api"
/ "tests", REPO_ROOT / "tests", ] for path in candidates: if path.exists() and path.is_dir():
return path return None
```

```
def run() -> int: start_dir = _discover_test_start_dir() if start_dir is None: print("WARN:
test directory not found, skip unittest discovery.") else: rel_start =
str(start_dir.relative_to(REPO_ROOT)).replace("\\", "/") test_cmd = [sys.executable, "-m",
"unittest", "discover", "-s", rel_start, "-t", ".", "-v"] test_result = subprocess.run(test_cmd,
cwd=REPO_ROOT, check=False) if test_result.returncode != 0: print(f"WARN: unittest
discovery failed for '{rel_start}', continue with line-count gate.") count_cmd =
[sys.executable, "scripts/count_lines.py"] count_result = subprocess.run(count_cmd,
cwd=REPO_ROOT, check=False, capture_output=True, text=True) if
count_result.returncode != 0: return count_result.returncode line_count =
int(count_result.stdout.strip()) print(f"Total non-empty lines: {line_count}") if line_count <
3000: print("Code line requirement not met.") return 1 return 0
```

```
if name == "main": raise SystemExit(run())
```

scripts/seed_demo_data.py

```
"""Rebuild the local demo database with fresh seed data."""
```

```

from future import annotations

import json import sys from pathlib import Path

REPO_ROOT = Path(file).resolve().parents[1] API_ROOT = REPO_ROOT / "apps" /
"api" sys.path.insert(0, str(API_ROOT))

from app.core.seed import seed_database # noqa: E402

def main() -> None: summary = seed_database(force=True) print(json.dumps(summary,
ensure_ascii=False, indent=2))

if name == "main": main()

```

scripts/soft_copyright_validate.py

-- coding: utf-8 --

""" Strict validator for software copyright (软著) materials.

Design goals:

- No third-party dependencies (stdlib only).
- Fail-fast with clear reasons and file pointers.
- Enforce the full rule set derived from the user's provided text:
 - triple consistency (name/version/modules/features)
 - document anti-template / anti-hallucination checks
 - sensitive info leakage checks
 - "feature described must be locatable in code" checks (string-based)

Usage: python -X utf8 .\workspace\scripts\soft_copyright_validate.py --project- root

E:\copyRight\workspace --name 项目名 --code-root

E:\copyRight\workspace\some_project

Notes:

- This script does not attempt to compute "similarity percentage" (no official algorithm).
- "Locatable in code" is enforced by requiring explicit evidence markers in docs and verifying that those markers appear in the codebase text. """

```
from future import annotations
```

```
import argparse import os import re import sys from dataclasses import dataclass from  
pathlib import Path from typing import Iterable, List, Optional, Sequence, Tuple
```

```
@dataclass(frozen=True) class Finding: level: str # "ERROR" | "WARN" message: str file:  
Optional[Path] = None line: Optional[int] = None
```

```
NAME_VERSION_RE = re.compile(r"(?P.+?)(?PV\d+(?:\d+)*)")
```

```
def _read_text(path: Path) -> str: return path.read_text(encoding="utf-8", errors="replace")
```

```
def _iter_files(root: Path, exts: Sequence[str]) -> Iterable[Path]: for p in root.rglob("*"): if  
p.is_file() and p.suffix.lower() in exts: yield p
```

```
def _find_first(pattern: re.Pattern[str], text: str) -> Optional[re.Match[str]]: return  
pattern.search(text)
```

```
def _line_no(text: str, pos: int) -> int: return text.count("\n", 0, pos) + 1
```

```
def _extract_softname_version(doc_text: str) -> Tuple[Optional[str], Optional[str]]: m =  
_find_first(NAME_VERSION_RE, doc_text) if not m: return None, None name =  
m.group("name").strip() version = m.group("version").strip() # Normalize common  
separators name = re.sub(r"[\u3000\t]+$", "", name) return name, version
```

```
def detect_sensitive_markers(text: str) -> List[str]: hits: List[str] = [] patterns = [  
r"https?:/", r"bwww.", r"b(?:C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z):\  
r"Users\  
r"b(author|created\sat|updated\sat|timestamp)\b", r"[A-Za-z0-9.%+-]+@[A-  
Za-z0-9.-]+\.[A-Za-z]{2,}", ] for pat in patterns: if re.search(pat, text,  
flags=re.IGNORECASE): hits.append(pat) return hits
```

```
def _detect_template_talk(text: str) -> List[str]: # Strongly discouraged phrases per rule  
set. phrases = [ "界面简洁", "操作简单", "功能强大", "性能优越", "首先", "其次", "最  
后", "综上所述", "总而言之", "易用", "先进", "高效", ] hits = [] for ph in phrases: if ph in  
text: hits.append(ph) return hits
```

```
EVIDENCE_RE = re.compile( r"(?:代码定位证据|代码位置|对应代码|实现位置)[:~\s*  
(?P.+)", flags=re.IGNORECASE, )
```

```
def _extract_evidence_lines(text: str) -> List[Tuple[int, str]]: out: List[Tuple[int, str]] = []  
for m in EVIDENCE_RE.finditer(text): ln = _line_no(text, m.start()) ev =
```

```
m.group("evidence").strip() # Trim trailing punctuation / markdown ev = ev.strip("` \t")
out.append((ln, ev)) return out
```

```
def _evidence_tokens(evidence: str) -> List[str]: # Evidence may contain: paths, symbols,
routes, endpoints, filenames. # Split by common separators while keeping tokens that can
be searched in code. raw = re.split(r"[ ,; \s]+", evidence) tokens: List[str] = [] for t in
raw: t = t.strip() if not t: continue # Drop very short noise tokens. if len(t) < 3: continue
tokens.append(t) # Deduplicate while keeping order seen = set() uniq: List[str] = [] for t in
tokens: if t in seen: continue seen.add(t) uniq.append(t) return uniq
```

```
def _search_tokens_in_code(code_root: Path, tokens: Sequence[str]) -> List[str]: # Search
tokens in common text code/config files. Ignore node_modules/dist/build outputs. exts = {
".ts", ".tsx", ".js", ".jsx", ".vue", ".py", ".java", ".go", ".cs", ".rb", ".php", ".rs", ".kt", ".kts",
".json", ".yaml", ".yml", ".toml", ".xml", ".md", ".sql", ".env", ".ini", ".properties", }
deny_dirs = {"node_modules", "dist", "build", "out", ".next", ".nuxt", "target", ".git"}
```

```
def is_denied(p: Path) -> bool:
    parts = set(p.parts)
    return any(d in parts for d in deny_dirs)
```

```
cache: List[Tuple[Path, str]] = []
for p in _iter_files(code_root, tuple(exts)):
    if is_denied(p):
        continue
    try:
        cache.append((p, _read_text(p)))
    except OSError:
        continue
```

```
missing: List[str] = []
for tok in tokens:
    found = False
    for _, txt in cache:
        if tok in txt:
            found = True
            break
    if not found:
        missing.append(tok)
return missing
```

```
def validate( materials_root: Path, a_name: str, code_root: Path, require_docx_pdf: bool, ) -
> List[Finding]: findings: List[Finding] = []
```



```

a_dir = materials_root / a_name
if not a_dir.exists():
    return [Finding("ERROR", f"Materials directory not found: {a_dir}")]

manual_md = a_dir / f"{a_name}操作手册.md"
code_md = a_dir / f"{a_name}源代码文档.md"
form_md = a_dir / "软件著作权登记申请表.md"

required = [manual_md, code_md, form_md]
for p in required:
    if not p.exists():
        findings.append(Finding("ERROR", f"Missing required file: {p}",

p))

if require_docx_pdf:
    docx_pdf = [
        a_dir / f"{a_name}操作手册.docx",
        a_dir / f"{a_name}操作手册.pdf",
        a_dir / f"{a_name}源代码文档.docx",
        a_dir / f"{a_name}源代码文档.pdf",
        a_dir / "软件著作权登记申请表.docx",
    ]
    for p in docx_pdf:
        if not p.exists():
            findings.append(Finding("ERROR", f"Missing required file: {p}

", p))
img_dir = a_dir / "images"
if not img_dir.exists() or not any(img_dir.iterdir()):
    findings.append(Finding("ERROR", f"Missing or empty images directory: {img_dir}",
img_dir))

if any(f.level == "ERROR" and f.message.startswith("Missing required
file")) for f in findings):
    return findings

manual_txt = _read_text(manual_md)
code_txt = _read_text(code_md)
form_txt = _read_text(form_md)

# 1) Triple consistency: name + version
m_name, m_ver = _extract_softname_version(manual_txt)
c_name, c_ver = _extract_softname_version(code_txt)
f_name, f_ver = _extract_softname_version(form_txt)

# We accept that extraction may be imperfect; but then it must be

```

```
explicitly provided. if not (m_name and m_ver): findings.append(Finding("ERROR",
"Cannot extract 软件名称/版本号 from 操作手册.md (expect '软件全称 V1.0'
somewhere).", manual_md)) if not (c_name and c_ver):
findings.append(Finding("ERROR", "Cannot extract 软件名称/版本号 from 源代码文
档.md (expect '软件全称 V1.0' somewhere).", code_md)) if not (f_name and f_ver):
findings.append(Finding("ERROR", "Cannot extract 软件名称/版本号 from 申请表.md
(expect '软件全称 V1.0' somewhere).", form_md))
```

```
if m_ver and not m_ver.startswith("V"):
    findings.append(Finding("ERROR", f"Version in 操作手册.md must start
```

```
with 'V': got {m_ver!r}", manual_md)) if c_ver and not c_ver.startswith("V"):
findings.append(Finding("ERROR", f"Version in 源代码文档.md must start with 'V': got
{c_ver!r}", code_md)) if f_ver and not f_ver.startswith("V"):
findings.append(Finding("ERROR", f"Version in 申请表.md must start with 'V': got
{f_ver!r}", form_md))
```

```
if m_name and c_name and f_name:
    if not (m_name == c_name == f_name):
        findings.append(
            Finding(
                "ERROR",
                f"软件名称不一致: 手册={m_name!r}, 源码文档={c_name!r},
                申请表={f_name!r}", )) if m_ver and c_ver and f_ver: if not (m_ver == c_ver == f_ver):
findings.append( Finding( "ERROR", f"版本号不一致: 手册={m_ver!r}, 源码文档=
{c_ver!r}, 申请表={f_ver!r}", ))
```

2) Sensitive info leakage checks

```
for p, txt in [(manual_md, manual_txt), (code_md, code_txt), (form_md,
form_txt)]: hits = _detect_sensitive_markers(txt) if hits:
findings.append(Finding("ERROR", f"Sensitive marker(s) detected: {' '.join(hits)}", p))
```

3) Anti-template / AI-ish talk checks

```
for p, txt in [(manual_md, manual_txt), (form_md, form_txt)]:
    hits = _detect_template_talk(txt)
    # Strict by default: treat as ERROR, but allow user to demote via
```

```
flag in future. if hits: findings.append(Finding("ERROR", f"Template/AI-ish phrases
detected: {' '.join(hits)}", p))
```

4) Evidence-required checks (anti-hallucination + code correspondence)

```
evidence_lines = _extract_evidence_lines(manual_txt)
```

```
if not evidence_lines:
```

```
    findings.append(
```

```
        Finding(
```

```
            "ERROR",
```

```
            "操作手册.md missing evidence markers. Add lines like
```

```
'代码定位证据: ' for each feature section.", manual_md, ) ) return findings
```

```
# Tokenize evidence and verify tokens appear in codebase.
```

```
all_missing: List[Tuple[int, str, str]] = []
```

```
for ln, ev in evidence_lines:
```

```
    toks = _evidence_tokens(ev)
```

```
    if not toks:
```

```
        findings.append(Finding("ERROR", "Empty/invalid evidence line
```

```
after '代码定位证据: ...", manual_md, ln)) continue missing =
```

```
_search_tokens_in_code(code_root, toks) for tok in missing: all_missing.append((ln, ev, tok))
```

```
if all_missing:
```

```
    sample = all_missing[:20]
```

```
    for ln, ev, tok in sample:
```

```
        findings.append(
```

```
            Finding(
```

```
                "ERROR",
```

```
                f'Evidence token not found in codebase: token={tok!r};
```

```
evidence_line={ev!r}", manual_md, ln, ) ) if len(all_missing) > len(sample):
```

```
findings.append(Finding("ERROR", f"... and {len(all_missing) - len(sample)} more  
missing evidence tokens.", manual_md))
```

```
return findings
```

```
def main(argv: Sequence[str]) -> int: ap = argparse.ArgumentParser() ap.add_argument("--  
project-root", required=True, help="Root folder that contains the {A}/ materials  
directory.") ap.add_argument("--name", required=True, help="需求名称 {A} (also  
materials subdirectory name).") ap.add_argument("--code-root", required=True,  
help="Codebase root used for 'locatable in code' checks.") ap.add_argument("--require-  
docx-pdf", action="store_true", help="Also require docx/pdf and images/ existence.") ns =  
ap.parse_args(argv)
```

```

materials_root = Path(ns.project_root)
a_name = ns.name
code_root = Path(ns.code_root)
require_docx_pdf = bool(ns.require_docx_pdf)

findings = validate(materials_root, a_name, code_root, require_docx_pdf)
errors = [f for f in findings if f.level == "ERROR"]
warns = [f for f in findings if f.level == "WARN"]

def fmt(f: Finding) -> str:
    loc = ""
    if f.file:
        loc = str(f.file)
    if f.line:
        loc += f": {f.line}"
    loc += ": "
    return f"[{f.level}] {loc} {f.message}"

for f in findings:
    print(fmt(f))

if errors:
    print(f"FAIL: {len(errors)} error(s), {len(warns)} warning(s)")
    return 2
print(f"PASS: {len(warns)} warning(s)")
return 0

if name == "main": raise SystemExit(main(sys.argv[1:]))

```